

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

NIVEL 4 (INTEGRADOR / PROFESIONAL)

Curso 4: Taller Práctico & Proyecto Final Personalizado

«Construcción de Soluciones Reales: Full-Stack Web, Chatbots de Atención y Sistemas de Gestión ACID»

Manual completo del curso compilado oficialmente. Incluye fundamentación teórica, metáforas conceptuales, diagramas de arquitectura, código fuente comentado, resolución de gotchas y bibliografía de estudio.

Instructor Principal: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Total de Módulos / Clases: 3 | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura del Manual de Curso

Este libro unificado contiene el desglose secuencial de cada módulo formativo, estructurado con rigor de ingeniería y diseñado para servir como manual de referencia permanente.

Plan de Estudios y Hoja de Ruta

El curso está dividido en las siguientes unidades temáticas secuenciales:

Código	Unidad y Metáfora	Objetivo Principal
Track 01	Track 01: Aplicaciones Web con Python (FastAPI & Streamlit) «El Restaurante: La Carta (Frontend) y la Cocina de Alta Eficiencia (Backend)»	Comprender la separación de responsabilidades Cliente-Servidor, APIs RESTful y e...
Track 02	Track 02: Chatbot Inteligente para Atención al Cliente «El Recepcionista Omnicanal y el Manual de Operaciones»	Comprender la gestión del estado conversacional, el enrutamiento de intenciones ...
Track 03	Track 03: Sistema de Gestión con Base de Datos Relacional «El Archivo Notarial y la Bóveda de Datos ACID»	Comprender el modelo relacional de datos, las claves primarias/foráneas y la int...



Criterio de Progresión

Cada unidad construye sobre los cimientos de la anterior. Se recomienda dominar la teoría, analizar el diagrama de arquitectura y replicar el código de cada sección antes de avanzar a la siguiente.

| Track 01: Aplicaciones Web con Python (FastAPI & Streamlit)

Una aplicación web desacoplada divide la presentación visual del procesamiento central de datos mediante contratos de comunicación HTTP (APIs REST).

☀ **Metáfora Central: El Restaurante: La Carta (Frontend) y la Cocina de Alta Eficiencia (Backend)**

El frontend (Streamlit) es la carta elegante y el mozo que atiende al comensal en la mesa. El backend (FastAPI) es la cocina profesional donde los chefs procesan las comandas con máxima higiene, rapidez y orden, entregando los platos listos en formato JSON.

Principios Teóricos y Modelo Mental

FastAPI: Framework moderno, basado en Starlette y Pydantic, con soporte nativo de asincronía (ASGI) y tipado estático.

Verbos HTTP Semánticos: GET (consultar datos), POST (crear nuevos registros), PUT (actualizar), DELETE (eliminar).

⚡ **Regla de Oro en Python**

Nunca mezcles lógica pesada de base de datos en el cliente visual; el cliente solo consume y renderiza.

Diagrama de Flujo Full-Stack: Streamlit <-> FastAPI <-> DB

Comunicación asíncrona mediante peticiones HTTP/JSON y validación cruzada.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	El usuario interactúa con widgets en Streamlit y presiona un botón.	Evento en UI
2. Evaluación	Streamlit envía una petición HTTP POST /api/v1/recurso con payload JSON.	Request sobre HTTP
3. Transformación	FastAPI valida los datos con Pydantic, ejecuta la lógica y persiste en DB.	Validación & Persistencia
4. Retorno / Salida	FastAPI responde HTTP 201 Created y Streamlit actualiza la vista reactivamente.	UI actualizada

Visualización Mental

FastAPI genera automáticamente documentación Swagger interactiva en la ruta /docs para probar todos tus endpoints.

Backend FastAPI con Endpoint RESTful Tipado

main.py (Python 3.10+)

PEP 8 Compliant

```
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel

app = FastAPI(title="API de Gestión de Productos", version="1.0.0")

class ProductoDTO(BaseModel):
    nombre: str
    precio: float
    categoria: str

db_productos: list[dict] = []

@app.post("/productos", status_code=status.HTTP_201_CREATED)
async def crear_producto(prod: ProductoDTO):
    nuevo = {"id": len(db_productos) + 1, **prod.model_dump()}
    db_productos.append(nuevo)
    return {"mensaje": "Producto creado", "data": nuevo}

@app.get("/productos")
async def listar_productos():
    return {"total": len(db_productos), "productos": db_productos}
```

⚠ Gotcha Frecuente (Trampa de Principiante)

Olvidar configurar el middleware CORS (Cross-Origin Resource Sharing), bloqueando las peticiones del frontend.

❌ Antipatrón

Sin configuración CORS: Streamlit o React no podrán consumir la API

✅ Patrón Correcto

```
from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(CORSMiddleware, allow_origins=
['*'])
```

| Track 02: Chatbot Inteligente para Atención al Cliente

Un chatbot empresarial no solo charla: responde preguntas frecuentes con precisión quirúrgica, consulta el estado de pedidos y transfiere a humanos cuando es necesario.

☀ **Metáfora Central: El Recepcionista Omnicanal y el Manual de Operaciones**

El chatbot es como el recepcionista estrella de una empresa: saluda cordialmente, recuerda todo lo que le dijiste en la conversación actual (memoria de sesión) y consulta de inmediato el manual de operaciones antes de dar una respuesta oficial.

Principios Teóricos y Modelo Mental

Gestión de Sesión: Cada usuario tiene un `session_id` único asociado a su buffer de historial en memoria (o en Redis).

Guardrails y System Prompt: Delimitan estrictamente las fronteras temáticas del bot para evitar que hable de temas ajenos a la empresa.

⚡ **Regla de Oro en Python**

Instruye siempre al chatbot en su System Prompt para que admita honestamente si no conoce una respuesta en lugar de inventar información.

Diagrama de Flujo Conversacional y Webhooks

Ciclo de vida del mensaje desde la app de mensajería hasta la síntesis de respuesta.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	El usuario envía un mensaje en Telegram/WhatsApp; la plataforma emite un Webhook HTTP.	Mensaje entrante
2. Evaluación	El Session Manager recupera el historial previo del usuario desde la memoria caché.	Historial de diálogo cargado
3. Transformación	Se inyecta el contexto RAG de la empresa y el LLM formula la respuesta corporativa.	Inferencia contextualizada
4. Retorno / Salida	Se guarda el nuevo turno en el historial y se envía el mensaje al canal del usuario.	Respuesta entregada al chat

Visualización Mental

Mantén los prompts del sistema concisos y enfócate en el tono de voz (amable, formal, conciso).

Motor de Chatbot con Historial de Sesión

main.py (Python 3.10+)

PEP 8 Compliant

```
class ChatbotAtencionCliente:
    def __init__(self, nombre_empresa: str):
        self.nombre_empresa = nombre_empresa
        self.sesiones: dict[str, list] = {}
        self.system_prompt = f"Eres el asistente virtual de {nombre_empresa}. Sé conciso y formal."

    def responder_usuario(self, user_id: str, mensaje: str) -> str:
        if user_id not in self.sesiones:
            self.sesiones[user_id] = [{"role": "system", "content": self.system_prompt}]

        # Agregar turno del usuario
        self.sesiones[user_id].append({"role": "user", "content": mensaje})

        # Simulación de respuesta del LLM contextualizada
        respuesta = f"Hola, gracias por contactar a {self.nombre_empresa}. ¿En qué puedo ayudarte?"
        self.sesiones[user_id].append({"role": "assistant", "content": respuesta})

        return respuesta
```

⚠ Gotcha Frecuente (Trampa de Principiante)

No limitar el tamaño del historial acumulado; con el tiempo la conversación agota la ventana de contexto y eleva los costes innecesariamente.

✗ Antipatrón

Acumular cientos de mensajes sin podar el historial

✓ Patrón Correcto

Mantener solo los últimos K mensajes (Sliding Window Memory)

| Track 03: Sistema de Gestión con Base de Datos Relacional

La memoria RAM se borra al apagar la computadora; una base de datos relacional garantiza que la información de tus clientes y finanzas persista para siempre de forma atómica e íntegra.

☀ **Metáfora Central: El Archivo Notarial y la Bóveda de Datos ACID**

Una base de datos relacional es como una bóveda notarial de alta seguridad: cada tabla es un libro de registros con columnas estrictas, y cada transacción es un contrato firmado. O se realizan todos los pasos de la operación o se cancela por completo sin dejar inconsistencias a medias.

Principios Teóricos y Modelo Mental

Propiedades ACID: Atomicidad (todo o nada), Consistencia (cumple reglas), Aislamiento (conurrencia segura), Durabilidad (persiste en disco).

Inyección SQL: La vulnerabilidad #1 en bases de datos; ocurre al concatenar texto crudo en queries. Se previene siempre con consultas parametrizadas (?) o (%s).

⚡ **Regla de Oro en Python**

NUNCA uses f-strings para construir sentencias SQL (ej: `f'SELECT * FROM u WHERE id={id}'`); usa siempre queries parametrizadas con tuplas.

Diagrama de Capas: Aplicación <-> Repositorio <-> Motor SQL

Arquitectura en capas (Layered Architecture) para aislar las sentencias SQL de la lógica de negocio.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	La capa de negocio solicita guardar o consultar una entidad.	Llamada a método del Repositorio
2. Evaluación	El Repositorio abre una conexión/cursor y prepara la sentencia parametrizada.	Preparación de la query
3. Transformación	El motor de base de datos ejecuta la transacción y valida claves únicas.	Ejecución ACID en disco
4. Retorno / Salida	Se realiza commit() para asegurar los cambios y se cierra la conexión de forma segura.	Datos persistidos permanentemente

Visualización Mental

Utiliza siempre context managers (with sqlite3.connect(...) as conn:) para asegurar el cierre de conexiones.

Repositorio de Datos Seguro con SQLite

main.py (Python 3.10+)

PEP 8 Compliant

```
import sqlite3

class RepositorioUsuarios:
    def __init__(self, db_path: str = "app.db"):
        self.db_path = db_path
        self._crear_tabla()

    def _crear_tabla(self) -> None:
        with sqlite3.connect(self.db_path) as conn:
            cursor = conn.cursor()
            cursor.execute('''
                CREATE TABLE IF NOT EXISTS usuarios (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    nombre TEXT NOT NULL,
                    email TEXT UNIQUE NOT NULL,
                    saldo REAL DEFAULT 0.0
                )
            ''')
            conn.commit()

    def insertar_usuario(self, nombre: str, email: str, saldo: float) -> int:
        with sqlite3.connect(self.db_path) as conn:
            cursor = conn.cursor()
            # Consulta parametrizada segura contra Inyección SQL
            cursor.execute(
                "INSERT INTO usuarios (nombre, email, saldo) VALUES (?, ?, ?)",
                (nombre, email, saldo)
            )
            conn.commit()
            return cursor.lastrowid
```

⚠ Gotcha Frecuente (Trampa de Principiante)

Concatenar variables de usuario directamente dentro de sentencias SQL, permitiendo ataques de Inyección SQL.

❌ Antipatrón

```
cursor.execute(f"SELECT * FROM users WHERE user = '{user}'") # ¡Vulnerable!
```

✅ Patrón Correcto

```
cursor.execute("SELECT * FROM users WHERE user = ?", (user,)) # Inmune a inyección
```

Conclusiones Generales del Curso

Has completado el recorrido integral de **Curso 4: Taller Práctico & Proyecto Final Personalizado**. El dominio de estos conceptos te sitúa con una ventaja competitiva sólida para el desarrollo de software profesional e Inteligencia Artificial.

Hitos Alcanzados

Comprensión profunda de la arquitectura, asimilación de patrones de diseño idiomáticos en Python, resolución de problemas algorítmicos y capacidad de integración de proyectos reales.

Fuentes Bibliográficas Canónicas

Recurso	Descripción	Enlace
Documentación Oficial Python	Especificación canónica del lenguaje	docs.python.org/3/
PEP 8 - Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones profesionales de ingeniería	realpython.com
Suite wisrovi en GitHub	Librerías open source de alto rendimiento	github.com/wisrovi

Agradecimiento del Instructor

Gracias por tu disciplina, curiosidad y compromiso en cada lección. Continúa pedaleando y construyendo software con propósito. ¡Nos vemos en el siguiente nivel!